Coventry University (CU) | National Institute of Business Management (NIBM)

NB5004CEM - Cybersecurity

Authentication: Passwords, Sessions & Tokens

Session 4 - IAM Layer, Part 1 of 2

Niranga Dharmaratna | niranga@nibm.lk

S4 Authentication: Passwords, Sessions & Tokens

Session Overview

Who are you? Proving identity at runtime.

- **Evaluate password storage, session, and token mechanisms against the Authenticity property *(LO1)***
- **Apply systematic reasoning to identify authentication weaknesses at the IAM layer *(LO3)***

> **Analogy**
> Today's spine: the Authenticity property (S1), enforced at runtime by authentication.

Authentication vs Authorisation: Today's Boundary

- **Authentication ("AuthN") answers: who are you? Today's entire scope.**
- **Authorisation ("AuthZ") answers: what are you allowed to do? That's Session 5 - RBAC, least privilege, MFA choice.**
- **The two fail independently - weak authentication, weak authorisation, or both. Today we only diagnose the first.**

Analogy

Authentication is showing ID at the door. Authorisation is which rooms your pass unlocks once inside. Today we only study the ID check.

Today in Numbers

4
Password stages
plaintext -> MD5 -> salted hash -> bcrypt/Argon2

4
Cookie flags
Secure, HttpOnly, SameSite, Expiry

4
JWT failure modes
none-alg, no exp, weak secret, exposed payload

Mechanism	Layer	Property Enforced
Password hashing	Application + Data	Confidentiality of the secret -> enables Authenticity
Session cookies	Network + Application	Authenticity of the ongoing session
JWT	Application	Integrity + Authenticity of the token

KEY CONCEPT

Authenticity Is The Property. Authentication Is The Mechanism.

S1 named Authenticity as a security property. S3 named the crypto mechanisms - signatures and certificates - that enforce it. S4 names the runtime mechanism: authentication. When authentication fails - weak passwords, no MFA, session hijacking - Authenticity fails. This is also the primary defence against Spoofing (S2's STRIDE).

What a Password Proves

- **A password is a shared secret proving identity at login - its Confidentiality is what makes the Authenticity claim credible.**
- **If the secret is exposed, anyone holding it can authenticate as the real user.**
- **Storage spans two layers: Application logic, Data layer for the hash.**

Analogy

A password is like a house key copied to a lockbox. A weak lockbox means anyone who opens it can walk in as you.

Password Storage: The Evolution

- **Plaintext: password stored as typed - any database read exposes every credential.**
- **Unsalted MD5: hash stored, but identical passwords always produce identical hashes - visible pattern, crackable with precomputed tables.**
- **Salted SHA-1 / SHA-256: a unique salt per user defeats precomputed tables, but the hash is still fast to compute.**
- **bcrypt / Argon2 / scrypt: deliberately slow, salted by design - the current standard for password storage.**

Fast Hashes vs Slow Hashes: Why Speed Is the Wrong Feature

Fast Hashes - Wrong for Passwords

MD5 / SHA-256

Designed for speed - useful for checking huge files quickly. That same speed lets an attacker try billions of password guesses per second once the hash is stolen.

Slow Hashes - Right for Passwords

bcrypt / Argon2 / scrypt

Deliberately slow by design, with a tunable cost factor. The same billions-of-guesses attack now takes years, not hours - because each guess is expensive, not just each dataset.

Salting Defeats Rainbow Tables

- A salt is random data added to a password before hashing, unique per user, stored alongside the hash.
- Without a salt, an attacker precomputes a rainbow table once and cracks every matching hash instantly.
- Failure beat: S3 named MD5/SHA-1 for passwords as a misuse - fast + unsalted means precomputed and cracked.
- Hook (IAM-01): passwords must be stored using a salted, slow hash. Test: inspect stored credentials. Feeds B2.

Sessions: Proving Identity Across Requests

- **HTTP is stateless - a cookie carries a session identifier so the server recognises the same user on the next request.**
- **The property at stake is Authenticity of the ongoing session - is this still really the user who logged in?**
- **Layer: Network for transmission, Application for how the cookie is used and protected.**

Analogy

A cookie is like a wristband at a venue. It proves you were checked at the door - but a copied or stolen wristband gets someone else in as you.

Cookie Security Flags: One Flag, One Attack

Flag	Prevents	Layer
Secure	Interception over unencrypted connections	Network
HttpOnly	XSS-based session theft (JS cannot read cookie)	Application
SameSite	Cross-site request forgery (CSRF)	Application
Expiry	Indefinitely valid sessions after compromise	Application

Cookie Hook: SRS Requirement

- **Testable requirement, IAM-01 (authentication-scoped): "Session cookies must set Secure, HttpOnly, and SameSite, with an explicit expiry."**
- **Acceptance test: inspect the Set-Cookie header on the login response - a missing flag fails the check.**
- **Failure beat: a cookie missing HttpOnly is readable by any injected script (A03/XSS, covered in S6) - session theft without ever touching the password.**
- **Feeds B2 (identity, authn/z, secrets) in Part B.**

Try It: Cookie Flag Toggler

- Open the Cookie Flag Toggler demo on your own device.
- Turn Secure, HttpOnly, and SameSite on and off, one at a time.
- Watch the attacker's-eye panel - which attack does each flag block, and which stays open when it's missing?
- This rehearses the DevTools lab task on a safe, simulated cookie before you inspect a real one.

JSON Web Tokens: header.payload.signature

- **A JWT has three parts: header (algorithm + type), payload (claims), signature (proves the token wasn't altered).**
- **Header and payload are base64-encoded, NOT encrypted - readable by anyone who has the token.**
- **The signature enforces Integrity of the token and Authenticity of the issuer.**

Analogy

A JWT is like a sealed but transparent envelope. Anyone can read what's inside (base64) - the wax seal (signature) proves it wasn't tampered with.

KEY CONCEPT

Base64 Is Not Encryption

Base64 is an encoding, not a cipher - it has no secret key, and decoding it is a one-line operation for anyone. A JWT payload is readable by anyone who intercepts it. Never place secrets - passwords, personal data, session keys - inside a JWT payload.

Four Ways a JWT Fails

Failure	What Goes Wrong
None-algorithm attack	alg header set to "none" - some libraries accept the token without checking any signature
Missing exp	Token has no expiry - stays valid forever if stolen
Weak or hardcoded secret	Signature can be recomputed or forged by an attacker who guesses or finds the secret
Sensitive data in payload	Base64 is readable, not secret - anyone with the token reads the claims

JWT Hook: SRS Requirement

- **Testable requirement, IAM-01 (authentication-scoped): "Tokens must specify and enforce a signing algorithm (never accept alg:none), include an exp claim, and never carry unencrypted sensitive data in the payload."**
- **Acceptance test: submit a token with alg set to none, or an expired exp - the server must reject both.**
- **Feeds B2 (identity, authn/z, secrets).**

Try It: JWT Anatomy + None-Attack Decoder

- Open the JWT demo and paste or edit a sample token.
- Watch header.payload.signature decode live - notice the payload is fully readable.
- Flip the algorithm to none and watch signature verification fall over.
- Today's showstopper insight: readable payload + forgeable signature = broken Authenticity.

One Property, Three Mechanisms

- **Password hashing, session cookies, and JWTs are three mechanisms - all enforcing the same property: Authenticity.**
- **Each has its own failure mode, but every failure tells the same story: the system can no longer trust who it's talking to.**
- **This is the evaluative lens the coursework rewards: not "is there a password field", but "does this actually enforce Authenticity?"**

Activity: Password Archaeology

- Five code snippets show credential handling across different languages.
- 1. Rank the five from most to least dangerous.
- 2. Name the specific flaw in each snippet.
- 3. Identify which security property is violated by each flaw.
- 4. State what SRS requirement would have prevented it.
- Reading and critiquing only - no writing code.

Small Lab: Cookie Inspection

- Open the provided deliberately vulnerable demo app.
- In DevTools, Application tab, inspect the session cookie.
- Identify which security flags are missing.
- What could an attacker do with a cookie missing HttpOnly or Secure?
- Document findings, each mapped to a security property and a layer.
- Optional: relate one finding to your assigned coursework domain.

Coursework Recap: What's Ready, What's Next

S4 Enables Now

Authentication SRS + B2 draft

Testable authentication requirements - password storage, cookie flags, token validation - each with an ID and acceptance test. Directly feeds B2 (identity, authn/z, secrets).

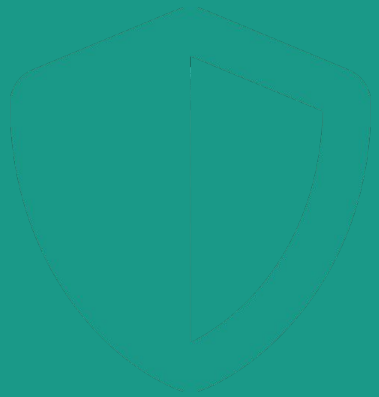
S5 Completes

RBAC, least privilege, MFA methods

IAM-02 (RBAC), least privilege analysis, privilege escalation paths, and comparative MFA methods (TOTP / push / FIDO2) complete the IAM layer next session.

Questions & Answers

Next: Session 5 - Authorisation, MFA & Least Privilege



Disclaimer & Copyrights

The information presented in this presentation is intended for general informational purposes only. It is not meant to be comprehensive or to constitute professional advice. Any reliance you place on such information is strictly at your own risk. While I strive to keep the information accurate and up-to-date, I do make no representations or warranties of any kind, express or implied, about the completeness, accuracy, reliability, suitability, or availability concerning the content within this presentation.

This presentation may include references or links to third-party websites or content. I do not endorse the views expressed or the information provided on such external sites. I am not responsible for the content of any linked site or any link contained in a linked site.

Any action you take upon the information presented in this presentation is strictly at your own discretion. I will not be liable for any losses or damages in connection with the use of this information.

This disclaimer is subject to change without notice.

Last modified: 2 Jul 2026